



가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 09 - City KPIs and Integration Lab

Chris Ohk

utilForever@gmail.com

- 10월 9일에 Lecture 10의 강의 영상이 업로드될 예정입니다.
- 10월 10일에 Lecture 11의 강의 영상이 업로드될 예정입니다.
- 10월 9일에 중간 발표에 대한 안내 사항을 공지할 예정입니다.

- Lecture 8의 도시 지표를 기존 도시 프로그램에 적용한다.
- 6개의 함수를 작성해 지표 계산 / 기록 / 저장 기능을 완성한다.
 - 작성 → 단계 검사 → 화면 확인 → 조건을 바꿔 실험

- 압축을 푼 스켈레톤 폴더에서 프로그램을 실행한다.
 - Windows: `py -m tkinter → py app.py`
 - macOS / Linux: `python3 -m tkinter → python3 app.py`
- `metrics.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

- 제공된 코드에서 TODO 주석 부분을 채운다.

파일	역할	수업 중 수정
metrics.py	지표 계산 / 기록	직접 작성
city_data.py / traffic.py 등	도시 / 도로망 / 교통 코드	제공
kpi_support.py	통근 자료 생성 / 갱신 / 파일 입출력	제공
app.py / kpi_gui.py	실행 / 지도와 지표 화면	제공
checks.py	단계별 자동 검사	실행

첫 화면 살펴보기

- 도시 지도 / 지표 / 시간에 따른 변화를 한 화면에서 확인한다.

도시 지표

0분 / 기록 1개

기본 도시

혼잡 도시

단절 도시

도시 편집

도로망과 교통

CSV 내보내기

상태 저장

불러오기

통근 기준 4.0 분 혼잡 기준: 대기 5 대 이상 유입 8 대/분 처리 용량 5 대/분 대기 차량 0 대

설정 / 도시 / 관찰 도로를 바꾸면 기존 기록을 지우고 0분부터 시작합니다.

도시 지표

시간에 따른 변화

0

1

2

3

4

0

1

2

3

물

H6 / 10

P공원

A4 / 10

J6 / 12

물

F8 / 20

붉은 테두리: 관찰 도로 / 초록 점: 개발 가능한 셀

H / A: 주택 / J: 상점 / F: 공장 / P: 공원

선택한 셀 (1, 2): 도로 / 이 도로의 대기열을 관찰합니다.

도시 현황과 통근 지표

지표	값	측정 기준
거주 인구	10명	주택에 거주하는 전체 주민
건물 수	4동	주택 / 상점 / 공장, 공원 제외
통근 접근성	100%	전체 주민 중 4분 이내에 도달
도달 불가	0명	상점 J까지 이어지는 경로가 없음
평균 통근 시간	2.2분	도달 가능한 주민만 포함
중앙값	3분	통근 시간을 정렬했을 때의 가운데 값
P90	3분	도달 가능한 통근의 90%가 이 시간 이내
개발 가능 토지	2셀	보전 구역을 제외한 도로 인접 빈 땅
관찰 시간	0분	0분 이후 진행한 관찰 구간
혼잡 시간	0분	구간 시작 시 대기 5대 이상
최대 대기	—	관찰한 대기 차량 수의 최댓값
현재 대기	0대	선택 도로의 현재 대기 차량 수

관찰 도로 (1, 2) / 현재 대기 0대

모든 주민이 상점 J로 한 번씩 통근한다고 가정합니다.
도달할 수 없는 주민은 평균에서 제외하고 별도로 셉니다.
1분 진행할 때 선택한 도로의 대기열만 갱신합니다.

기본 도시를 불러왔습니다. 지표를 확인하고 1분씩 진행해 보세요.

- 도시 자료에서 지표를 계산하고, 시각과 함께 기록한다.

자료	형태	저장하는 내용
city / transport	딕셔너리	지도와 건물 / 시각과 도로 상태
trips	딕셔너리의 리스트	주민 1명당 예상 이동 시간
observations	딕셔너리의 리스트	관측 구간 길이 / 대기 차량 수
kpis / history	딕셔너리 / 리스트	현재 지표 / 시각별 지표 기록

이번 실습의 도시

- 지난주 지도에 주택 A를 추가했다. 인구는 10명이다.

y / x	0	1	2	3	4
0	빈 땅	보전	도로	빈 땅	물
1	H / 6명	공원 P	A / 4명	상점 J	물
2	도로	도로	도로	도로	빈 땅
3	도로	도로	도로	도로	공장 F

실습 1 : 도시 규모

- `summarize_city(city)`를 작성한다.
 - 주택의 거주 인구를 더하고, 공원을 제외한 건물 수를 센다.
 - 반환값 : `population / building_count`

거주 인구와 건물 수

- 주택 2동의 인구는 10명 / 전체 건물은 4동이다.

시설	종류	인구 집계	건물 집계
H	주택	6명	1동
A	주택	4명	1동
P	공원	제외	제외
J / F	상점 / 공장	제외	2동

- `metrics.py`의 TODO 1을 완성해 보자.
 - `python checks.py --stage 1`
 - 기본 도시 : 인구 10명 / 건물 4동
- 건물 목록이 비어 있다면 두 값은 각각 얼마일까?

- 주택의 인구를 더하고 공원을 제외한 건물 수를 센다.

```
def summarize_city(city):  
    population = 0  
    building_count = 0  
  
    for building in city["buildings"].values():  
        if building["kind"] == "house":  
            population += building["population"]  
        if building["kind"] != "park":  
            building_count += 1  
  
    return {  
        "population": population,  
        "building_count": building_count  
    }
```

실습 2 : 통근 지표

- `summarize_trips(trips, limit)`를 작성한다.
 - `trips`에는 주민 1명당 통근 기록이 1건씩 들어 있다.
 - `time_min`이 `None`이면 목적지에 도달할 수 없다.
- 인원 집계 / 접근성 비율 / 평균 이동 시간을 반환한다.

주민 수를 반영한 평균

- 접근성 100% / 평균 예상 이동 시간 2.2분

출발 주택	예정 인원	1명당 시간	시간 합
H	6명	3분	18분
A	4명	1분	4분
합계	10명	4분 이내 10명	22분

도달하지 못한 주민의 처리

- 도달 불가능 접근성의 분모에 포함하고 평균에서는 제외한다.

입력 상황	접근성	평균 이동 시간
6명 도달 불가 / 4명 1분	$4 \div 10 = 40\%$	$4 \div 4 = 1\text{분}$
10명 모두 도달 불가	$0 \div 10 = 0\%$	계산할 시간 없음 / None
예정 통근 0건	분모 없음 / None	분모 없음 / None

- `metrics.py`의 TODO 2를 완성해 보자.
 - `python checks.py --stage 2`
 - 기본 도시 : 접근성 1.0 / 평균 2.2분
- 0분과 None을 같은 조건으로 처리하면 어떤 문제가 생길까?

통근 지표 해설 : 인원과 시간

- 시간이 있는 기록에서 인원과 시간 합을 구한다.

```
def summarize_trips(trips, limit):  
    reachable = 0  
    within_limit = 0  
    total_time = 0  
  
    for trip in trips:  
        time = trip["time_min"]  
  
        if time is not None:  
            reachable += 1  
            total_time += time  
  
            if time <= limit:  
                within_limit += 1  
  
    planned = len(trips)
```

통근 지표 해설 : 비율과 평균

- 같은 자료라도 지표에 따라 분모가 다르다.

```
# 접근성은 전체 예정 인원, 평균 시간은 도달 가능한 인원을 기준으로 한다.  
return {  
    "planned_trips": planned,  
    "reachable_trips": reachable,  
    "unreachable_trips": planned - reachable,  
    "within_limit_trips": within_limit,  
    "accessibility_ratio":  
        within_limit / planned if planned else None,  
    "mean_time_min":  
        total_time / reachable if reachable else None,  
}
```

- `find_developable_cells(city, protected)`를 작성한다.
 - 도로 / 건물 / 공원이 없는 육지에서 보전 셀을 제외한다.
 - 상하좌우에 도로가 하나 이상 있는 셀을 찾는다.
- 조건을 만족한 셀의 (x, y) 를 리스트로 반환한다.

빈 땅에 개발 조건 적용하기

- 결과 : (3, 0), (4, 2) / 총 2셀

빈 육지	보전 대상	도로 인접	개발 가능
(0, 0)	아니요	아니요	불가
(1, 0)	예	예	불가
(3, 0)	아니요	예	가능
(4, 2)	아니요	예	가능

직접 해보기 : 개발 가능 토지

가상공간과 메타버스
City KPIs and Integration Lab

- `metrics.py`의 TODO 3을 완성해 보자.
 - `python checks.py --stage 3`
 - 기본 도시 : $[(3, 0), (4, 2)]$ / 총 2셀
- 한 셀이 도로 두 곳과 맞닿아 있다면 몇 번 추가해야 할까?

- 제외 조건을 먼저 검사한 뒤 도로와 맞닿았는지 확인한다.

```
def find_developable_cells(city, protected):  
    result = []  
  
    for y, row in enumerate(city["grid"]):  
        for x, cell in enumerate(row):  
            if (cell["terrain"] != "land" or cell["road"]  
                or cell["building_id"] is not None  
                or (x, y) in protected):  
                continue  
  
            for nx, ny in neighbors4(city, x, y):  
                if city["grid"][ny][nx]["road"]:  
                    result.append((x, y))  
                    break  
  
    return result
```

- `summarize_congestion(observations, threshold)`를 작성한다.
 - 입력은 관측 구간의 길이와 대기 차량 수다.
 - 대기 차량이 5대 이상인 구간의 길이를 더한다.
- 전체 관측 시간 / 혼잡 시간 / 관측 최댓값을 반환한다.

구간별 대기 차량과 혼잡 시간

- 혼잡 3분 / 관측 최댓값 12대 / 현재 대기 15대

관측 구간	구간 길이	대기 차량	혼잡 여부
[0, 1)	1분	0대	아니요
[1, 2)	1분	3대	아니요
[2, 3)	1분	6대	예
[3, 4)	1분	9대	예
[4, 5)	1분	12대	예

- `metrics.py`의 TODO 4를 완성해 보자.
 - `python checks.py --stage 4`
 - 5분 관측 결과 : 혼잡 3분 / 관측 최댓값 12대
- 혼잡한 구간의 길이가 2분이라면 합계에 얼마를 더할까?

혼잡 시간 해설 : 구간 합산

- 혼잡한 구간의 수가 아니라 구간의 길이를 더한다.

```
def summarize_congestion(observations, threshold):  
    observed = 0  
    congested = 0  
    peak = None  
  
    for observation in observations:  
        duration = observation["duration_min"]  
        queue = observation["queue"]  
        observed += duration  
  
        if queue >= threshold:  
            congested += duration  
  
        if peak is None or queue > peak:  
            peak = queue
```

- 관측 전에는 시간은 0분이고 최댓값은 None이다.

```
return {  
    "observed_minutes": observed,  
    "congested_minutes": congested,  
    "peak_queue": peak,  
}
```

- `append_history(history, minute, kpis)`를 작성한다.
 - 현재 지표에 시각을 넣어 `history`에 추가한다.
 - 같은 시각은 한 번만 기록하고, 과거 값은 유지한다.
- 추가하면 `True` / 같은 시각이 이미 있으면 `False`를 반환한다.

- 시작 시점 1행 + 1~5분의 5행 = 총 6행

순서	한 번의 1분 진행
1	구간 시작의 대기 차량 수를 관측 기록에 추가
2	제공 코드로 교통 상태를 갱신
3	갱신한 상태에서 도시 지표를 계산
4	새 시각의 지표를 이력에 한 번 추가
5	지도 / 현재 지표 / 이력 화면을 갱신

- deepcopy는 딕셔너리 안의 리스트까지 새로 복사한다.

단계	현재 지표 kpis	저장할 기록 row
복사 직후	개발 가능 셀 [(3, 0)]	개발 가능 셀 [(3, 0)]
현재 목록만 비움	개발 가능 셀 []	개발 가능 셀 [(3, 0)]

- `metrics.py`의 TODO 5를 완성해 보자.
 - `python checks.py --task 5`
 - 새 시각이면 1행 추가 / 같은 시각이면 이력 유지
- 기록한 뒤 `kpis` 안의 리스트를 바꿔도 과거 값이 유지될까?

- 중복 시각을 먼저 검사하고, 독립된 기록을 추가한다.

```
def append_history(history, minute, kpis):  
    for row in history:  
        if row["minute"] == minute:  
            return False  
  
    row = deepcopy(kpis)  
    row["minute"] = minute  
    history.append(row)  
  
    return True
```

실습 6 : 도시 상태 저장

- `make_snapshot(...)`을 작성한다.
 - 도시 / 교통 / 지표 이력 / 설정 / 관측값을 한데 모은다.
 - 깊이 복사한 스냅샷을 딕셔너리로 반환한다.
- 파일 저장과 불러오기는 제공 코드가 처리한다.

- 현재 상태와 이후 계산에 필요한 자료를 함께 저장한다.

항목	저장할 내용
city	지도 / 건물 / 도시 규칙
transport	현재 시각 / 대기열 / 도로 비용
history	지금까지의 지표 이력
settings	통근 기준 / 혼잡 기준 / 보전 셀 / 목적지
observations	선택 도로의 관측 구간과 대기 차량 수

직접 해보기 : 도시 상태 저장

가상공간과 메타버스
City KPIs and Integration Lab

- `metrics.py`의 TODO 6을 완성해 보자.
 - `python checks.py --stage 6`
 - `format`과 다섯 입력 자료를 `deepcopy`로 복사해 반환한다.
- 저장한 뒤 원래 도시를 수정해도 스냅샷은 그대로일까?

- 저장할 상태 전체를 한 번에 깊이 복사한다.

```
def make_snapshot(  
    city, transport, history, settings, observations):  
    # 중첩된 자료도 복사해야 저장 시점의 상태가 이후 변화와 분리된다.  
    return deepcopy({  
        "format": "virtual-city-kpi-v1",  
        "city": city,  
        "transport": transport,  
        "history": history,  
        "settings": settings,  
        "observations": observations,  
    })
```

CSV 이력과 JSON 스냅샷

- CSV는 지표 이력 / JSON은 다시 시작할 도시 상태다.

목적	제공 함수 / 파일
지표 이력 내보내기	<code>export_history_csv(...)</code> / CSV
전체 상태 저장	<code>save_session(...)</code> / JSON
도시 다시 불러오기	<code>load_session(...)</code> / JSON
전체 연계 확인	<code>python checks.py --integration</code>

비교 실험 1 : 기본 도시

- 목표 : 주민의 80% 이상이 4분 이내에 목적지에 도달한다.

0분의 확인 항목	기본 도시
인구 / 건물	10명 / 4동
접근성 / 평균 시간	100% / 2.2분
도달 불가 / 개발 가능 토지	0명 / 2셀

비교 실험 2 : 같은 도시의 혼잡

- 혼잡 시나리오를 새로 열어 0분의 값을 비교한다.

지표	기본 / 0분	혼잡 / 0분
인구	10명	10명
4분 이내 접근성	100%	40%
평균 이동 시간	2.2분	3.4분
도달 불가 인원	0명	0명

비교 실험 3 : 도로가 끊기면

- 연결 끊김 시나리오에서는 주민 10명 모두 목적지에 갈 수 없다.

지표	기본 / 0분	연결 끊김 / 0분
4분 이내 접근성	100%	0%
평균 이동 시간	2.2분	계산할 수 없음
도달 불가 인원	0명	10명
개발 가능 토지	2셀	4셀

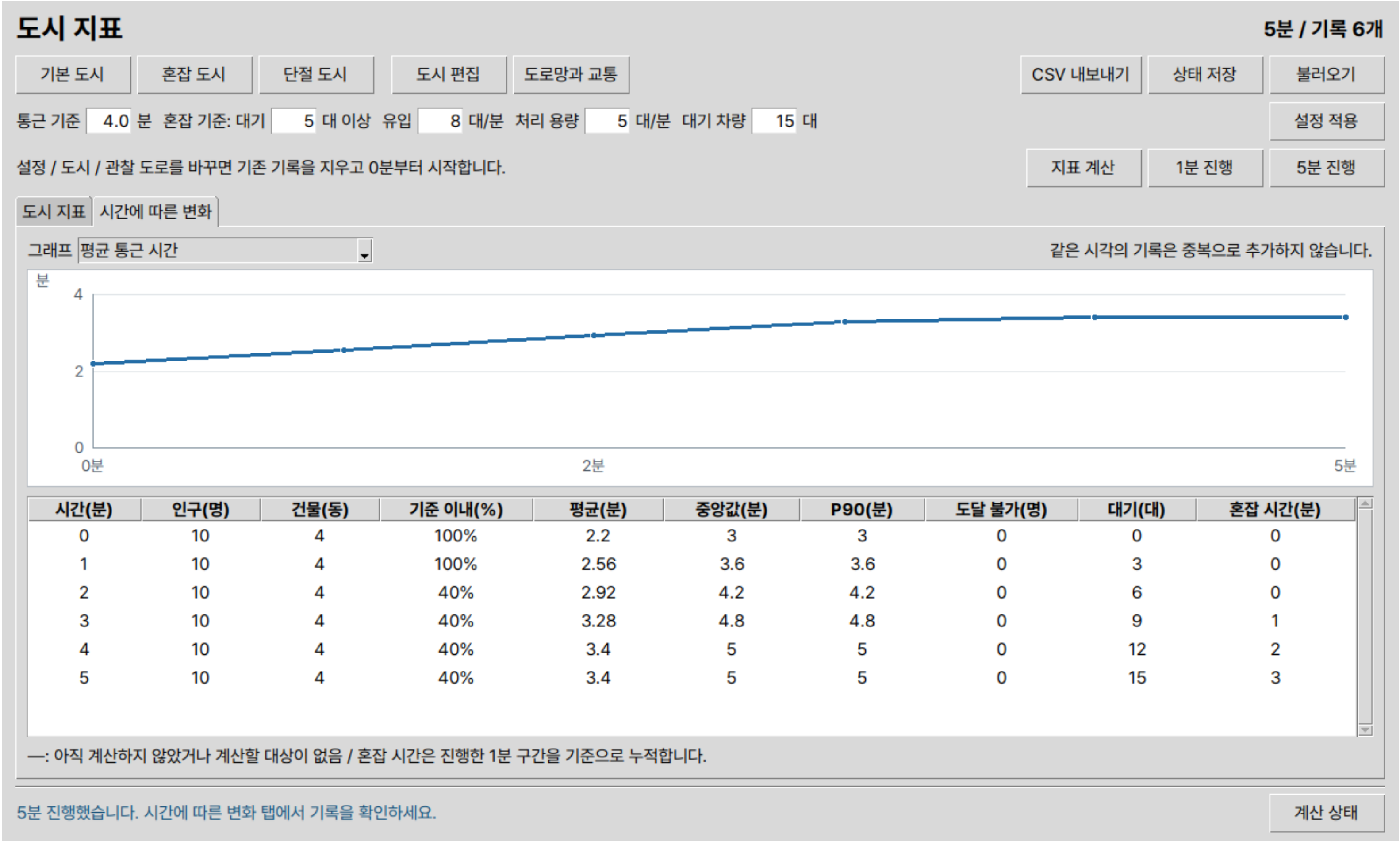
평균이 같은 두 통근 기록

- 별도 기록 X와 Y는 평균이 같지만 오래 걸리는 사람 수가 다르다.

기록	10명의 이동 시간	평균	중앙값	P90
X	4분 × 10명	4분	4분	4분
Y	1분 × 8명, 16분 × 2명	4분	1분	16분

비교 실험 4 : 5분 동안의 변화

- 현재 대기 15대 / 혼잡 3분 / 이력 6행을 확인한다.



CSV에 기록된 시계열

- CSV의 한 행은 한 시점의 도시 지표다.

minute	population	접근성 비율	평균 / 분	현재 대기
0	10	1.0	2.20	0
1	10	1.0	2.56	3
2	10	0.4	2.92	6
5	10	0.4	3.40	15

직접 해보기 : 저장한 도시 복원

가상공간과 메타버스
City KPIs and Integration Lab

- 기본 시나리오를 5분 진행한 뒤 JSON과 CSV를 저장한다.
 - 한 번 더 진행한 6분 결과를 적고 프로그램을 종료한다.
- 저장한 JSON을 불러와 5분 상태가 돌아오는지 확인한다.
 - 다시 1분 진행해 앞서 적은 6분 결과와 비교한다.

- python checks.py와 통합 검사가 모두 통과하는지 확인한다.

기능	확인할 결과
도시 규모	인구 10명 / 전체 건물 4동
통근 / 개발 토지	기본 평균 2.2분 / 접근성 100% / 2셀
혼잡 / 이력	기본 5분 후 혼잡 3분 / 이력 6행
스냅샷	복원 전후 같은 시각의 지표가 일치

- 도시 편집에서 도로 한 칸을 추가하거나 삭제한다.
 - 도시 적용을 누른 뒤 새 기록의 0분 값을 확인한다.
- 접근성 / 평균 이동 시간 / 개발 가능 토지의 변화를 비교한다.
 - 예상과 다른 값이 나오면 해당 함수의 입력과 조건을 확인한다.

- 건물과 통근 자료를 집계해 도시 지표를 계산했다.
- 관측 구간으로 혼잡 시간을 구하고, 시각별 지표를 기록했다.
- 평균과 함께 접근성 / 도달 불가 / 분포를 비교했다.
- 도시 상태를 저장하고 불러와 시뮬레이션을 이어서 실행했다.

감사합니다.

utilForever@gmail.com

<https://github.com/utilForever>

Facebook, Twitter, Instagram, LinkedIn: utilForever